

Teoría de la Computabilidad

Módulo 6: Lenguajes no libres de contexto

Departamento de Cs. e Ing. de la Computación
Universidad Nacional del Sur
Bahía Blanca, Argentina

Propiedades de LLC

Análogamente a lo analizado para lenguajes regulares, veremos un Segundo teorema de 'pumping' que ayuda a ver si un Lenguaje es LC o no.

- **Primer teorema de Pumping:** sirve para demostrar que un lenguaje NO ES regular.
- **Segundo teorema de Pumping:** sirve para demostrar que un lenguaje NO ES libre de contexto.

Conceptos previos - fan-out

Sea $G=(V_n, V_t, S, P)$ una GLC. El **fan-out** de G es el mayor número de símbolos que aparece en el lado derecho de cualquier regla de G , esto es:

$$\phi(G) = \max\{ \text{long}(\beta) \mid A \rightarrow \beta \in P \}$$

$G=(V_n, V_t, S, P)$, $V_n=\{S, R\}$, $V_t=\{b, a\}$,
 $P=\{S \rightarrow \lambda, S \rightarrow R, R \rightarrow aRb, R \rightarrow ab\}$

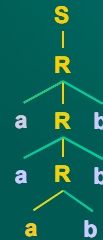
$$\phi(G) = 3$$

$G=(V_n, V_t, Q, P)$, $V_n=\{Q, S\}$, $V_t=\{1, 0\}$,
 $P=\{Q \rightarrow 0, Q \rightarrow 1, S \rightarrow 00, Q \rightarrow 011, S \rightarrow 11, Q \rightarrow 11Q0S, S \rightarrow 0SQ0\}$

$$\phi(G) = 5$$

Conceptos previos - camino

Un **camino (path)** en un árbol de derivación es una secuencia de nodos, cada uno conectado con otro por un segmento: el primer nodo es la raíz, y el último una hoja.



La **longitud** del camino es el número de segmentos que lo componen.

Camino: **SRa**

Longitud del camino = **2**

Cant. Nodos involucrados = **3**

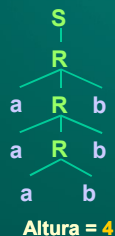
Camino: **SRRRa**

Longitud del camino = **4**

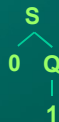
Cant. Nodos involucrados = **5**

Conceptos previos - altura del árbol

La **altura** de un árbol de derivación es la longitud del camino más largo.



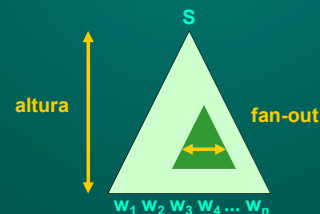
Altura = **4**



Altura = **2**

Conceptos previos

Se puede establecer una relación entre la **altura del árbol**, el **fan-out** y la **longitud de la cadena** que produce...

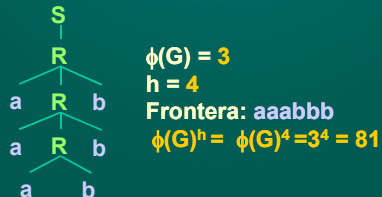


Si sabemos la altura del árbol, y el máximo número de hijos que un nodo puede tener, podemos acotar la longitud de la frontera...

Conceptos previos

Lema: La frontera w de cualquier árbol de derivación de G de altura h tiene una longitud menor o igual a $\phi(G)^h$

$G=(V_n, V_t, S, P)$, $V_n=\{S, R\}$, $V_t=\{b, a\}$,
 $P=\{S \rightarrow \lambda, S \rightarrow R, R \rightarrow aRb, R \rightarrow ab\}$



Conceptos previos

Corolario:

El árbol de derivación de $w \in L(G)$, con $|w| > \phi(G)^k$, debe tener un camino con longitud mayor que k .

$G=(V_n, V_t, S, P)$, $V_n=\{S, R\}$, $V_t=\{b, a\}$,
 $P=\{S \rightarrow \lambda, S \rightarrow R, R \rightarrow aRb, R \rightarrow ab\}$

$\phi(G) = 3$

$w = \text{aaaaaaaaaaaaaaaaabbbbbbbbbbbbbbb}$

$|w| = 30$

$|w| > 3^3$

Por lo tanto, sin duda el árbol de derivación de w tiene una altura mayor que 3.

Teorema de Pumping para GLC

Teo: Sea $G=(V_n, V_t, S, P)$ una GLC. Ent. $\exists k$ tal que toda cadena $w \in L(G)$ de longitud mayor a k puede reescribirse como $w=uvxyz$, tal que $vy \neq \lambda$, y $uv^nxy^n z \in L$, para todo $n \geq 0$ y se satisface que $|vxy| \leq k$.

Demostración:

Sea $w \in L(G)$, $|w| > k$ y sea T un árbol para w , con raíz en S , y con frontera w que tiene el menor nro. posible de nodos entre todos los árboles con igual raíz e igual frontera.



Teorema de Pumping para GLC

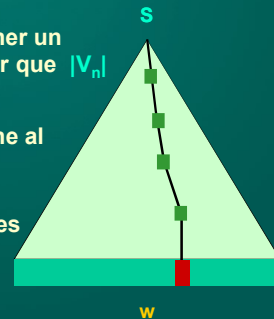
Sea $k = \phi(G)^{|V_n|}$ tal que $|w| > k$.

Como $|w| > \phi(G)^{|V_n|}$, debe tener un camino con longitud mayor que $|V_n|$ (por corolario).

Entonces este camino tiene al menos $|V_n| + 2$ nodos

Sólo uno de estos nodos es terminal, el resto no terminales.

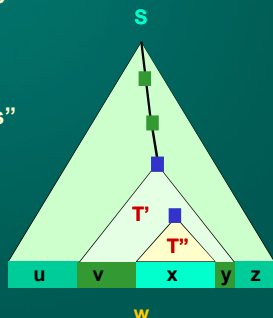
Es decir, al menos $|V_n| + 1$ no terminales existen en el camino



Teorema de Pumping para GLC

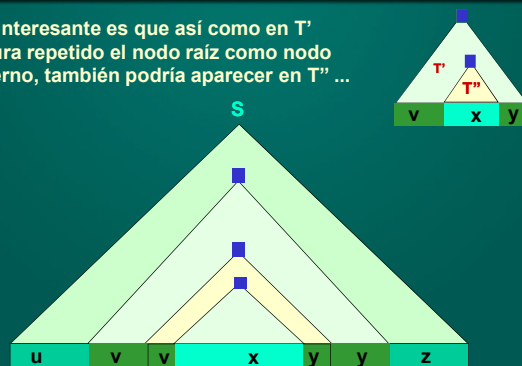
Algún no terminal se repite en el camino...

Podemos distinguir entonces cinco "porciones" de la cadena w ...

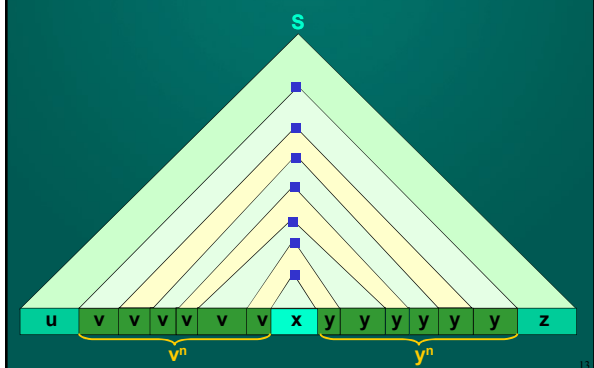


Teorema de Pumping para GLC

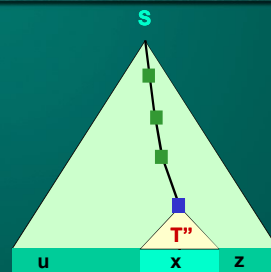
Lo interesante es que así como en T' figura repetido el nodo raíz como nodo interno, también podría aparecer en T'' ...



Teorema de Pumping para GLC

Y así sucesivamente n veces....

Teorema de Pumping para GLC

Resta considerar por qué exigimos $vy \neq \lambda$.Si $vy = \lambda$, ent. existe un árbol con raíz S y frontera w con menos hojas que T .Pero esto viola la suposición de que T era el árbol más chico posible de esta clase.Luego debe satisfacerse que $vy \neq \lambda$.

Utilidad del teorema - ejemplos

En forma similar a como lo hacíamos para los lenguajes regulares, el teorema nos sirve para demostrar que un lenguaje **no es libre de contexto**.

La estrategia es similar: probarlo por el absurdo utilizando el teorema.

EJEMPLO:

Demostrar que el lenguaje

$$L = \{a^n b^n c^n | n \geq 0\}$$

no es libre de contexto.

Ejemplo - $L = \{a^n b^n c^n | n \geq 0\}$ Supongamos que L es libre de contexto.Sea k el número del que habla el teorema.Sea $w = a^k b^k c^k$. Evidentemente $|w| > k$, por lo que se debería cumplir el teorema con esta cadena w .Según este teorema, w se puede dividir en $uvxyz$ (con $vy \neq \lambda$ y $|vxy| \leq k$) de alguna forma tal que $uv^nxy^n z$ pertenece a L para todo $n \geq 0$.Busquemos entonces si existe esta forma de dividir la cadena w ...

$$a^k b^k c^k = \underbrace{aa..aabb..bbcc..cc}_{uvxyz}$$

 $uvxyz$ v e y tienen apariciones del mismo símbolo:

	u	v	x	y	z
1) tienen "a"	a^{p1}	a^{p2}	a^{p3}	a^{p4}	$a^{p5} b^k c^k$
tienen "c"	$a^k b^{p1}$	b^{p2}	b^{p3}	b^{p4}	$b^{p5} c^k$
tienen "b"	$a^k b^k c^{p1}$	c^{p2}	c^{p3}	c^{p4}	c^{p5}

Con $p1+p2+p3+p4+p5=k$, $p2+p4 > 0$ y $p2+p3+p4 \leq k$ $uv^2xy^2z \notin L$ por que tiene mayor cantidad de uno de los símbolos que de los otros dos.

$$\text{En 1) } uv^2xy^2z = a^{p1} a^{2(p2)} a^{p3} a^{2(p4)} a^{p5} b^k c^k \\ = a^{k+p2+p4} b^k c^k \notin L$$

$$a^k b^k c^k = \underbrace{aa..aabb..bbcc..cc}_{uvxyz}$$

 $uvxyz$ v e y tienen un solo tipo de símbolo, pero diferentes entre sí:

	u	v	x	y	z
2) "a" y "b"	a^{p1}	a^{p2}	$a^{p3} b^{q1}$	b^{q2}	$b^{q3} c^k$
"b" y "c"	$a^k b^{p1}$	b^{p2}	$b^{p3} c^{q1}$	c^{q2}	c^{q3}
"a" y "c"	a^{p1}	a^{p2}	$a^{p3} b^{q1} c^{q2}$	c^{q3}	

 $\rightarrow |vxy| > k$ Con $\Sigma p_i = k$, $\Sigma q_i = k$, $p2+q2 > 0$ y $p2+p3+q1+p2 \leq k$ $uv^2xy^2z \notin L$ por que tiene mayor cantidad de dos de los símbolos que del restante.

$$\text{En 2) } uv^2xy^2z = a^{p1} a^{2(p2)} a^{p3} b^{q1} b^{2(q2)} b^{q3} c^k \\ = a^{k+p2} b^{k+q2} c^k \notin L$$

$$a^k b^k c^k = aa..aabb..bbcc..cc$$

uvxyz

v o y tienen mas de un tipo de símbolo:

	u	v	x	y	z
3) “a” y “b”	a^{p1}	$a^{p2} b^{q1}$	b^{q2}	b^{q3}	$b^{q4} c^k$
“a” y “b”	a^{p1}	a^{p2}	a^{p3}	$a^{p4} b^{q1}$	$b^{q2} c^k$
“b” y “c”	$a^k b^{p1}$	$b^{p2} c^{q1}$	c^{q2}	c^{q3}	c^{q4}
“b” y “c”	$a^k b^{p1}$	b^{p2}	b^{p3}	$b^{p4} c^{q1}$	c^{q2}

$uv^2xy^2z \notin L$ ya que tiene símbolos mezclados: tiene apariciones de b's antes que a's o apariciones de c's antes que b's.

$$\text{En 3) } uv^2xy^2z = a^{p1} (a^{p2} b^{q1})^2 b^{q2} (b^{q3})^2 b^{q4} c^k \\ = a^{p1} a^{p2} b^{q1} a^{p2} b^{q1} b^{q2} b^{2q3} b^{q4} c^k \notin L$$

Las restantes formas de particionar w no respetan las condiciones $vy \neq \lambda$ o $|vxy| \leq k$

(Ejercicio: identificar las particiones restantes y verificar que condición no respetan)

Conclusión:

Hemos demostrado que no existe forma de particionar $w = a^k b^k c^k$ en subcadenas uvxyz, de forma que $uv^n xy^n z \in L$ para todo $n \geq 0$.

Por lo tanto L no puede ser libre de contexto. Absurdo que provino de suponer que $L = \{a^n b^n c^n | n \geq 0\}$ es LC. Luego, L no es LC, cqd.

Dos Ejercicios Relacionados

Probar (utilizando pumping lema) que

$$L_1 = \{a^n b^m a^n b^m | n, m > 1\}$$

no es libre de contexto.

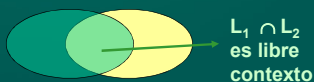
Probar que

$$L_2 = \{ww | w \in \{a,b\}^*\}$$

no es libre de contexto.

IDEA: Usar la propiedad de que los LC son cerrados bajo $\cap$ con LR.

L_1 (regular) L_2 (libre contexto)



Leng. de Programación que no son LC

¿Para qué estudiamos si un leng. L es LC o no?

Parte de la motivación radica en ver si un lenguaje de programación es LC o no.

Notemos que hay características de Pascal que escapan a las gramáticas LC. Por ejemplo, **que todo identificador tiene que ser declarado antes de ser utilizado.**

Debemos hacer hincapié también en la diferencia entre la **especificación** de un lenguaje y su **implementación**.

Ejemplo: Pascal no es Libre de Contexto

Consideremos un subconjunto de Pascal,

$L = \{ \text{program PRUEBA;} \\ \text{var } w:\text{boolean;} \\ \text{begin } w:=\text{true; end} \\ | \text{ } w \text{ es un ident. válido en Pascal} \}$

Sea LP el conjunto de todos los programas Pascal que son compilables y sintácticamente correctos.

Sea

$R = \{ \text{program PRUEBA;} \\ \text{var let(dig } \cup \text{ let)*: boolean;} \\ \text{begin let(dig } \cup \text{ let)* := true; end} \\ | \text{ dig}=\{0..9\} \text{ y let}=\{c,f,j\} \}$

Nota: elegimos un subconj. de las letras para simplificar la demostración

Ejemplo: Pascal no es LC

`program PRUEBA; var c:boolean; begin f:= true; end`

es un elemento de R, pero no de L.

Notemos que $L = LP \cap R$. Planteamos ent. el sgte. homomorfismo:

$$h(s) =_{\text{def}} \begin{cases} h(c) = c, & h(f) = f, & h(j) = j \\ h(\alpha) = \alpha, & \forall \alpha \in \{0..9\} \\ h(\beta) = \lambda, & \forall \beta \in \{c,f,j,0..9\} \end{cases}$$

Si aplico h(s) a L, ent. $h(L) = \{ww | w \in \{c,f,j,0..9\}^*\}$. R es regular, y el tipo de LP es lo que queremos determinar.

El rol del homomorfismo h

$L = \{\text{program_PRUEBA; var_w:boolean; begin w:=true; end} \mid w \text{ es un ident. válido en Pascal}\}$



$s = \text{program_PRUEBA; var_cj3:boolean; begin cj3:=true; end}$ es tal que $s \in L$.

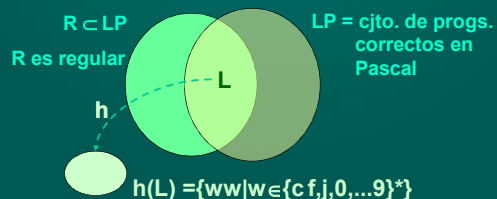


$h(s) = \{cj3cj3\}$

Ejemplo: Pascal no es LC

$L = \{\text{program PRUEBA; var w:boolean; begin w:=true; end} \mid w \text{ es un ident. válido en Pascal}\}$

$R = \{\text{program PRUEBA; var let(dig} \cup \text{let)*: boolean; begin let(dig} \cup \text{let)* := true; end} \mid \text{dig} = \{0..9\} \text{ y let} = \{c, f, j\}\}$



Ejemplo: Pascal no es LC

Supongamos por el absurdo que LP es libre de contexto.

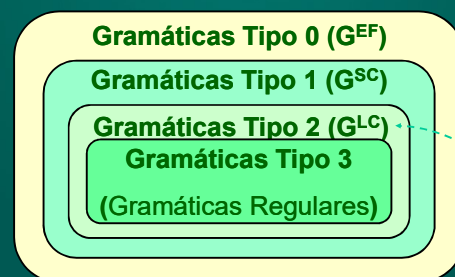
Por la propiedad de intersección antes vista, entonces, $LP \cap R$ es LC.

Pero como $L = LP \cap R$, entonces L sería LC. Pero como los lenguajes LC son cerrados bajo homomorfismo, también $h(L)$ es LC.

Pero el lenguaje $h(L) = \{ww \mid w \in \{c, f, j, 0, \dots, 9\}^*\}$ no es LC (ver ejercicio anterior).

El absurdo provino de suponer que LP era libre de contexto. Luego LP no es libre de contexto, cq.d.

Clasificación de gramáticas de Chomsky



Aplicamos Pumping

Gramáticas (repaso)

Recordemos....

Def.: Una **gramática estructurada por frases (GEF)** es una 4-upla $G = (V_n, V_t, S, P)$, donde

V_n = conj. finito de **símbolos no terminales** (o auxiliares)

V_t = conj. finito de **símbolos terminales**

S = **símbolo inicial**, $S \in V_n$

P = conj. finito de **reglas de producción** $\alpha \rightarrow \beta$.

Estudiaremos clases de gramáticas que surgen al tipificar las reglas de producción permitidas.

Gramática Sensible Al Contexto (GSC)

Def.: Sea una **gramática** $G = (V_n, V_t, S, P)$. Diremos que G es una **gramática sensible al contexto (GSC)** (o **Gramática Tipo 1**) si toda producción $\alpha \rightarrow \beta$ en P es tal que

$\text{long}(\alpha) \leq \text{long}(\beta)$,

$\alpha \in (V_n \cup V_t)^* V_n (V_n \cup V_t)^*$ y

$\beta \in (V_n \cup V_t)^*$

Veamos un ejemplo tradicional...

Ejemplo - $L = \{a^n b^n c^n | n \geq 0\}$

Vimos que

$$L = \{a^n b^n c^n | n \geq 0\}$$

no es libre de contexto. Intentemos generar una **gramática sensible al contexto (SC)** que genere L .

Utilizaremos reglas de producción con símbolos “viajeros” (esto es, símbolos auxiliares para garantizar el “equilibrio” en la derivación).

31

Ejemplo - $L = \{a^n b^n c^n | n \geq 0\}$

Definimos $G = (V_n, V_t, X_0, P)$,

$V_n = \{X_0, X_1, X_2\}$,

$V_t = \{a, b, c\}$,

$P = \{$
 $X_0 \rightarrow abc,$
 $X_0 \rightarrow a X_1 bc,$
 $X_1 b \rightarrow b X_1,$
 $X_1 c \rightarrow X_2 bcc,$
 $b X_2 \rightarrow X_2 b,$
 $a X_2 \rightarrow aa X_1,$
 $a X_2 \rightarrow aa \}$

X_0
 $\downarrow$
 aX_1bc
 $\downarrow$
 abX_1c
 $\downarrow$
 abX_2bcc
 $\downarrow$
 aX_2bbcc
 $\downarrow$
 $aabbcc$

32

Ejemplo - $L = \{ww | w \in \{a,b\}^*\}$

Definimos $G = (V_n, V_t, X_0, P)$,

$V_n = \{X_0, X_1, X_2, X_3, Y_0, Y_1\}$, $V_t = \{a, b\}$,

$P = \{$
 $X_0 \rightarrow X_1 X_2 X_3,$
 $X_1 X_2 \rightarrow b X_1 Y_1$
 $Y_0 b \rightarrow b Y_0$
 $Y_0 X_3 \rightarrow X_2 a X_3$
 $a X_2 \rightarrow X_2 a$
 $X_1 X_2 \rightarrow \lambda$
 $X_1 X_2 \rightarrow a X_1 Y_0$
 $Y_1 a \rightarrow a Y_1$
 $Y_1 b \rightarrow b Y_1$
 $Y_1 X_3 \rightarrow X_2 b X_3$
 $b X_2 \rightarrow X_2 b$
 $X_3 \rightarrow \lambda$
 $\}$

¿de qué tipo es esta gramática?

33

¿Existen autómatas para GSC?

- A diferencia de las gramáticas de tipo 3 y 2, no estudiaremos el autómata asociado en esta parte de la materia.
- El autómata que corresponde se denomina **Autómata Acotado Linealmente**, y se analizará en el próximo capítulo como restricción sobre **Máquinas de Turing**.

34

Resumen

- **Teorema de Pumping** para lenguajes libres de contexto
- Ejemplos de uso de **Teorema del Pumping** en lenguajes libres de contexto.
- Lenguajes de Programación que **no son LC** (ej: Pascal).
- Introducción a los **lenguajes tipo 1**: lenguajes sensibles al contexto (**LSC**).

35